

Documentação Arquitetural - S2: Regras Temporais e Eventos

1. Visão Geral do Componente

O componente **S2 (Regras Temporais e Eventos)** atua como o motor de purificação e consolidação de dados na **Camada 3 (Serviços)** do sistema. O seu propósito exclusivo é receber inferências brutas e ruidosas da Camada 2 (juntamente com os dados espaciais do componente S1), estabilizá-las temporalmente e unificá-las.

O S2 atua como um escudo protetor para a Camada 4 (Aplicação/A3): ele garante que o motor de decisão receba apenas eventos consistentes, eliminando flutuações de detecção, duplicações de câmera e falsos positivos de borda.

2. Princípios de Design Adotados

O desenvolvimento do S2 foi pautado em dois princípios fundamentais da Engenharia de Software:

- **Separação de Interesses (*Separation of Concerns*):** O S2 não toma decisões de negócio nem emite alertas. A sua responsabilidade limita-se à estabilização técnica dos dados (limpeza e deduplicação). A semântica do que é um "risco crítico" ou "situação normal" é delegada inteiramente ao componente A3.
- **Ocultação de Informação (*Information Hiding*):** Parâmetros críticos de configuração, como a janela de tempo do *debounce* e os limiares de tolerância espacial, estão encapsulados de forma privada nas classes (-`debounce_window_ms`). Isso garante baixo acoplamento e impede que componentes externos alterem o comportamento do filtro em tempo de execução.

3. Funcionalidades e Regras de Processamento

3.1. Filtro de Ruído Temporal (Janela e Debounce)

Detecções visuais na borda das zonas de interesse tendem a oscilar rapidamente entre "dentro" e "fora". O S2 implementa algoritmos de limitação de taxa (*sliding window / token bucket*) para aplicar um *debounce* a esses eventos. Atualizações excessivas no mesmo milissegundo são descartadas, garantindo uma taxa de atualização estável para o Dashboard.

3.2. Persistência Mínima (TTL e Condition Tracker)

Para conferir resiliência ao sistema contra falhas da Inteligência Artificial (como oclusões visuais temporárias, onde uma pessoa passa atrás de uma pilastra), o S2 utiliza um rastreador de condição acoplado a um *Time-To-Live* (TTL). Se uma entidade desaparece do fluxo de entrada, ela é mantida na memória do S2 até que o TTL expire. Isso evita o registro incorreto de múltiplos eventos de saída e reentrada para a mesma entidade.

3.3. Fusão e Deduplicação de Identidades

O **MotorEstadoConcorrente** resolve conflitos oriundos de múltiplas fontes (várias câmeras apontando para o mesmo local). Utilizando limiares de distância espacial e sincronia temporal, o sistema correlaciona detecções concorrentes e funde as identidades, emitindo um único `global_entity_id`.

3.4. Governança de Confiança e Dúvida Explícita

O S2 atua como o primeiro filtro de incerteza da IA, roteando os eventos com base no score de confiança recebido:

- **Score Alto (≥ 0.8):** Aprovado e consolidado automaticamente.
- **Score Baixo (< 0.5):** Descarte sumário (*drop*) para evitar lixo no sistema.
- **Dúvida Explícita (0.5 a 0.8):** A entidade é repassada no Payload Canônico com o status de "dúvida explícita", forçando o motor de regras da Camada 4 a solicitar revisão e intervenção de um Operador Humano.

4. Interface de Saída: O Evento Canônico

Após passar por todos os filtros e pelo motor de estado, o S2 constrói o **Evento Canônico**. Este DTO (*Data Transfer Object*) possui todos os seus atributos expostos publicamente para facilitar o consumo da Camada 4. O contrato estabelecido garante a entrega das seguintes informações estruturadas em JSON:

- ID Global Unificado (`global_entity_id`)
- Fontes Originárias (`fontes`)
- Pontuação de Confiança Consolidada (`pontuacao_confianca`)
- Status de Processamento (`status_final`)
- Timestamp da consolidação.

5. Estratégia de Simulação e Testes (Passthrough)

Durante as etapas de validação arquitetural (Sprints iniciais), o S2 é testado em modo isolado através de uma estratégia de *mocking*.

O script `gerar_mock.py` simula as camadas anteriores, gerando o arquivo `s1_evento_espacial`, que imita a entrada de dados crus com ruídos e duplicações intencionais. Ao processar este arquivo, o S2 aplica efetivamente suas lógicas de filtro e gera a saída `fusao_deduplicacao`. Essa abordagem prova o funcionamento das regras

temporais de ponta a ponta sem a necessidade de hardware físico ou de integração prematura com os algoritmos de visão computacional.